

Expliquer mon RAG simplement

Comment fonctionne la brique de recherche documentaire — version courte, sans jargon.

J'ai créé une première brique RAG pour un SaaS de gestion immobilière. L'objectif est de permettre à un utilisateur de poser une question sur ses documents — par exemple des contrats, des paiements ou des informations locataires — et d'obtenir une réponse basée sur les documents concernés.

Du document aux morceaux

D'abord, on importe un document et on extrait son texte. Comme un document complet peut être trop long et contenir plusieurs sujets, on le découpe en petits passages appelés **chunks**. Chaque chunk garde aussi des informations utiles, comme le document dont il provient, sa page ou sa date.

Des morceaux aux nombres

Ensuite, chaque chunk est transformé en **embedding**, aussi appelé représentation vectorielle. C'est un vecteur de nombres qui représente le sens du texte. Les nombres, pris séparément, ne veulent pas dire grand-chose : c'est l'ensemble du vecteur qui permet au modèle de rapprocher des textes qui parlent de la même idée.

EXEMPLE

Un chunk qui indique qu'un règlement de loyer n'a pas été reçu sera proche, en sens, d'une question comme : « Quels locataires n'ont pas encore payé ? », même si les mots employés ne sont pas exactement les mêmes.

On stocke alors chaque chunk, son embedding et ses métadonnées dans la base de données, avec un stockage vectoriel adapté à la recherche de similarité.

De la question à la réponse

Quand l'utilisateur pose une question à l'IA, la question est elle aussi transformée en embedding. On compare le vecteur de cette question avec les vecteurs de tous les chunks stockés. On récupère alors les quelques chunks les plus proches en sens : c'est la **recherche sémantique**.

Cette recherche peut être complétée par une recherche par mots-clés exacts, utile pour des éléments précis comme un nom de locataire, un numéro de bureau, une date ou un montant.

Enfin, on envoie uniquement les chunks les plus pertinents au modèle de génération, avec la question de l'utilisateur. Le modèle peut alors rédiger une réponse claire, fondée sur les informations retrouvées, au lieu d'essayer de répondre à partir de rien.

En résumé

1 Document → extraction du texte

On sort le texte brut du fichier PDF.

2 Texte → découpage en chunks

Des passages courts, compréhensibles isolément.

3 Chunks → embeddings et stockage

Chaque passage devient un vecteur de nombres, rangé avec ses métadonnées.

4 Question utilisateur → embedding

La question passe par le même modèle que les chunks.

5 Comparaison des vecteurs → chunks les plus pertinents

Similarité entre le vecteur de la question et ceux des chunks.

6 Chunks pertinents + question → réponse générée par l'IA

Le modèle rédige à partir des passages retrouvés, pas de sa mémoire.

Le choix technique que j'ai fait

Le choix que j'ai fait pour les embeddings est un modèle local avec `sentence-transformers`. Cela évite d'envoyer les données potentiellement sensibles des locataires vers un service externe, tout en restant gratuit pour le prototype.

Où en est le prototype

Le texte ci-dessus décrit le système **complet visé**. Voici ce qui tourne réellement aujourd'hui :

Étape	État	Précision
1 — Extraction du texte	Fonctionne	Avec <code>pypdf</code>
2 — Nettoyage	Fonctionne	Python pur
3 — Découpage en chunks	Fonctionne	Sans métadonnées pour l'instant
4 — Embeddings	Fonctionne	Modèle local, 384 nombres par chunk
5 — Recherche sémantique	Fonctionne	Vecteurs gardés en mémoire, pas encore en base
6 — Réponse générée	À faire	Nécessite un modèle de génération
Stockage en base de données	À faire	PostgreSQL + pgvector prévu
Recherche par mots-clés	À faire	En complément du sémantique
Intégration au SaaS	À faire	Django, upload, chat

Pourquoi ce prototype existe. Il est volontairement construit étape par étape, en Python simple, sans base de données ni framework. Chaque brique est testée seule avant d'être reliée aux autres. L'objectif n'est pas d'avoir un système en production, mais de comprendre précisément ce que fait chaque partie — avant de l'intégrer au SaaS où les enjeux de fiabilité et de confidentialité seront réels.